

PRACTICAL NO: 4

AIM:

1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

- Kill Processes by name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

2. Write a program for process creation using C

- Orphan Process
- Zombie Process

3. Create the process using fork () system call

- Child Process creation
- Parent Process creation
- PPID and PID THEORY:

This practical focuses on understanding process management in the Linux operating system. A process is a program in execution, and Linux manages multiple processes through multitasking. Processes are created using the fork() system call, which forms parent and child processes. Each process is identified using PID and PPID to maintain hierarchy. The experiment also demonstrates process execution, termination, and the use of system calls like exec() and wait(). It further explains special cases such as orphan and zombie processes in Linux

PERFORMANCE:

- Kill Processes by name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

COMMAND:

```

bhushan123@LAPTOP-0V4BCL4Q:~$ ps -ef
UID          PID    PPID  C  TIME TTY          TIME CMD
root           1        0  0  12:55 ?        00:00:00 /lib/systemd/systemd --system --deserialize 22
root           2          1  0  12:55 ?        00:00:00 /init
root           7          2  0  12:55 ?        00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --
root          61          1  0  12:55 ?        00:00:00 /lib/systemd/systemd-journald
root          88          1  0  12:55 ?        00:00:00 /lib/systemd/systemd-udev
systemd+     93          1  0  12:55 ?        00:00:00 /lib/systemd/systemd-resolved
systemd+     96          1  0  12:55 ?        00:00:00 /lib/systemd/systemd-timesyncd
root        187          1  0  12:55 ?        00:00:00 /usr/sbin/cron -f -P
message+    189          1  0  12:55 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --syst
root        194          1  0  12:55 ?        00:00:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
syslog      195          1  0  12:55 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
root        202          1  0  12:55 ?        00:00:00 /lib/systemd/systemd-logind
root        223          1  0  12:55 hvc0     00:00:00 /sbin/agetty -o -p -- \u --noclear --keep-baud console 115200,38400,
root        227          1  0  12:55 tty1     00:00:00 /sbin/agetty -o -p -- \u --noclear tty1 linux
root        229          1  0  12:55 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-s
root        291          2  0  12:55 pts/1    00:00:00 /bin/login -f
bhushan+    322          1  0  12:55 ?        00:00:00 /lib/systemd/systemd --user
bhushan+    323        322  0  12:55 ?        00:00:00 (sd-pam)
bhushan+    338        291  0  12:55 pts/1    00:00:00 -bash
root        765          1  0  12:58 ?        00:00:00 /usr/libexec/packagekitd
root        769          1  0  12:58 ?        00:00:00 /usr/libexec/polkitd --no-debug
root       1397          2  0  13:06 ?        00:00:00 /init
root       1398        1397  0  13:06 ?        00:00:00 /init
bhushan+   1403        1398  0  13:06 pts/0    00:00:00 -bash
bhushan+   1417        1403  0  13:06 pts/0    00:00:00 ps -ef
bhushan123@LAPTOP-0V4BCL4Q:~$ ps -ef |grep firefox
bhushan+    1423        1403  0  13:06 pts/0    00:00:00 grep --color=auto firefox
bhushan123@LAPTOP-0V4BCL4Q:~$ ps -ef | grep firefox
bhushan+    1427        1403  0  13:07 pts/0    00:00:00 grep --color=auto firefox
bhushan123@LAPTOP-0V4BCL4Q:~$ kill 9407
-bash: kill: (9407) - No such process
bhushan123@LAPTOP-0V4BCL4Q:~$ pkill firefox
bhushan123@LAPTOP-0V4BCL4Q:~$
```

```

bhushan123@LAPTOP-0V4BCL4Q:~$ ps -ef |grep firefox
bhushan+    1423        1403  0  13:06 pts/0    00:00:00 grep --color=auto firefox
bhushan123@LAPTOP-0V4BCL4Q:~$ ps -ef | grep firefox
bhushan+    1427        1403  0  13:07 pts/0    00:00:00 grep --color=auto firefox
bhushan123@LAPTOP-0V4BCL4Q:~$ kill 9407
-bash: kill: (9407) - No such process
```

```

bhushan123@LAPTOP-0V4BCL4Q:~$ ps -ef |grep firefox
bhushan+    1423        1403  0  13:06 pts/0    00:00:00 grep --color=auto firefox
bhushan123@LAPTOP-0V4BCL4Q:~$ ps -ef | grep firefox
bhushan+    1427        1403  0  13:07 pts/0    00:00:00 grep --color=auto firefox
bhushan123@LAPTOP-0V4BCL4Q:~$ kill 9407
-bash: kill: (9407) - No such process
bhushan123@LAPTOP-0V4BCL4Q:~$ pkill firefox
bhushan123@LAPTOP-0V4BCL4Q:~$
```

2. Write a program for process creation using C

Orphan Process:

An orphan process is a child process whose parent process terminates before the child finishes execution. The orphan process is adopted by the init or system process.

➤ orphan.c :

Output:

```
bhushan123@LAPTOP-0V4BCL4Q:~$ nano orphan.c
bhushan123@LAPTOP-0V4BCL4Q:~$ gcc orphan.c -o orphan
bhushan123@LAPTOP-0V4BCL4Q:~$ ./orphan
Parent exiting...
bhushan123@LAPTOP-0V4BCL4Q:~$ CHChild Process
PID = 1526
PPID = 1398 (Parent is init)
|
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child
        sleep(5);
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d (Parent is init)\n", getppid());
    } else {
        // Parent
        printf("Parent exiting...\n");
    }

    return 0;
}
```

- **Zombie Process**

A zombie process is a child process that has completed execution but still remains in the process table because its parent has not read its exit status.

➤ **Zombie.c :**

➤ **Output:**

```
bhushan123@LAPTOP-0V4BCL4Q:~$ nano zombie3.c
bhushan123@LAPTOP-0V4BCL4Q:~$ gcc zombie3.c -o zombie3
gcc: error: unrecognized command-line option '-o'
bhushan123@LAPTOP-0V4BCL4Q:~$ gcc zombie3.c -o zombie3
bhushan123@LAPTOP-0V4BCL4Q:~$ ./zombie3
Child exiting...
Parent still running...
bhushan123@LAPTOP-0V4BCL4Q:~$ |
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child
        printf("Child exiting...\n");
    } else {
        // Parent
        sleep(10); // Parent does not call wait()
        printf("Parent still running...\n");
    }

    return 0;
}
```

3. Create the process using fork () system call

- Child Process creation
- Parent Process creation
- PPID and PID

Process:

A process is an instance of a program that is currently being executed in the operating system. It includes program code, data, stack and system resources.

Process ID (PID):

Process ID (PID) is a unique numerical identifier assigned by the operating system to each running process for identification and management.

Parent Process:

A parent process is a process that creates one or more child processes using system calls such as fork().

Child Process:

A child process is a newly created process that is generated by a parent process and executes independently.

Parent Process ID (PPID):

PPID shows the process ID of the parent of a running process.

- fork_demo:
- Output:

```
bhushan123@LAPTOP-0V4BCL4Q:~$ nano fork_demo.c
bhushan123@LAPTOP-0V4BCL4Q:~$ gcc fork_demo.c -o fork_demo
bhushan123@LAPTOP-0V4BCL4Q:~$ ./fork_demo
Parent Process
PID = 1599
Child PID = 1600
Child Process
PID = 1600
PPID = 1398
bhushan123@LAPTOP-0V4BCL4Q:~$ |
```

```
GNU nano 0.2
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
    }
    else {
        // Parent process
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}|
```

4. Infinite loop process:

An infinite loop process is a process that runs continuously without termination until it is manually stopped by the user or system.

```
m309@m309-BY-OEM: $ nano loop.c
m309@m309-BY-OEM: $ gcc loop.c -o loop
m309@m309-BY-OEM: $ ./loop

Running...
Running...
Running...
Running...
Running...
Running...
```

```
GNU nano 7.2 loop.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    while (1) {
        printf("Running...\n");
        sleep(1);
    }
    return 0;
}
```

Stopped the infinite loop:

[illegible]

